

MODE.CPP & SIGNAL HANDLING — FULL EXPLANATION

PART 1 — mode.cpp overview

`mode.cpp` implements one IRC command, `MODE`, split across two functions:

```
❏  
  
handl_Mode(fd, msg)    <- validation, permission checks, dispatch  
ft_help_mode(...)     <- parses the mode string, applies changes,  
                        builds the broadcast line
```

This split matters: `handl_Mode` handles everything that can be checked *before* looking at individual flag characters (is the client registered? does the channel exist? are they a member? are they an operator?). Once all of that passes, it hands off to `ft_help_mode`, which walks the mode string character-by-character and does the actual work. Keeping these separate makes each function's job obvious and easier to test.

```
❏  
  
MODE <channel> [<+/-modes> [<args...>]]
```

Examples:

```
❏  
  
MODE #test      -> query current modes (324 reply)  
MODE #test +i   -> turn on invite-only  
MODE #test +o bob -> give bob operator status  
MODE #test +tk secret -> turn on topic-restriction AND set a key,  
                        in one command  
MODE #test -kl  -> remove the key and the user limit
```

PART 2 — handl_Mode, step by step

```
❏  
  
cpp  
void Server::handl_Mode(int fd, const Message &msg)  
{  
    if (!clients[fd].is_registered())  
    {  
        send_numeric_reply(fd, 451, "*", ":You have not registered");  
        return;  
    }  
}
```

Step 1 — registration gate. Same pattern as every other handler (JOIN, PRIVMSG, TOPIC...): an unregistered client can't do anything but PASS/NICK/USER.

```
❏  
  
cpp
```

```

if (msg.params.empty())
{
    send_numeric_reply(fd, 461, clients[fd].get_nick(), "MODE :Not enough parameters");
    return;
}

```

Step 2 — must at least name a channel. `MODE` with zero arguments is malformed.

```

cpp
const std::string &channel_name = msg.params[0];
Channel *channel = find_channel(channel_name);

if (!channel)
{
    send_numeric_reply(fd, 403, clients[fd].get_nick(), channel_name + " :No such channel");
    return;
}

```

Step 3 — the channel must exist. `find_channel` looks it up in the `channels` map and returns `NULL` if nothing's there. This `NULL` check is critical — a previous version of this code skipped it and dereferenced a null pointer, crashing the whole server on the very first `JOIN` to a brand-new channel. Always check a pointer before using it.

```

cpp
if (!channel->is_member(fd))
{
    send_numeric_reply(fd, 442, clients[fd].get_nick(),
        channel_name + " :You're not on that channel");
    return;
}

```

Step 4 — sender must be a member. You can't inspect or change modes on a channel you haven't joined.

```

cpp
if (msg.params.size() < 2)
{
    std::string modes = channel->get_mode_string();
    if (modes.empty())
        modes = "+";
    send_numeric_reply(fd, 324, clients[fd].get_nick(), channel_name + " " + modes);
    return;
}

```

Step 5 — query-only case. `MODE #chan` with no mode string means "tell me the current modes," not "change something." This check must come *before* the operator check below, because querying is allowed for any member, not just operators. (This exact ordering mistake happened with `TOPIC` earlier in this

project — the operator check ran before the query check, which incorrectly blocked non-operators from even *reading* the topic. Same principle applies here.)

```
cpp
if (!channel->is_operator(fd))
{
    send_numeric_reply(fd, 482, clients[fd].get_nick(),
        channel_name + " :You're not channel operator");
    return;
}
```

Step 6 — only operators may change modes. By this point we know it's an actual change request (there IS a mode string), so operator status is required.

```
cpp
const std::string &modestr = msg.params[1];
std::string mode_line = ft_help_mode(modestr, channel, msg, fd, channel_name);
if (!mode_line.empty())
    diffusion_msg_to_channel(*channel, -1, mode_line);
```

Step 7 — delegate and broadcast. `ft_help_mode` does the real work and returns either:

a fully-formed `MODE` line to broadcast (something changed), or

an empty string (nothing changed — either every flag failed, or the string was empty/garbage)

The `!mode_line.empty()` check only broadcasts when there's an actual change to report. Note the exact wording matters here — this was written backwards once (`if (mode_line.empty())`) which meant successful mode changes were silently applied server-side but NEVER announced to the channel. Always read a condition out loud against the case you expect to succeed: "if there IS a line, broadcast it" ->

```
!mode_line.empty().
```

PART 3 — ft_help_mode, the parsing loop

```
cpp
std::string Server::ft_help_mode(std::string modestr, Channel *channel,
    const Message msg, int fd, std::string channel_name)
{
    size_t arg_index = 2;
    char sign = '+';
    char last_sign_written = 0;
    std::string change;
    std::vector<std::string> change_args;
```

`arg_index` starts at 2 because `msg.params[0]` is the channel name and `msg.params[1]` is the mode string itself — the FIRST actual argument (a key, a nickname, a limit number) is `msg.params[2]`.

`sign` tracks whether we're currently in a `+` or `-` run. IRC mode strings can flip mid-string: `+o-l bob` means "+o" then "-l".

`change` / `change_args` accumulate what actually happened, so the final broadcast line matches reality — not just what was *requested* (a request can partially fail, e.g. `+ol` where `l`'s argument is missing but `o` still succeeded).

```
cpp
for (size_t i = 0; i < modestr.size(); i++)
{
    char c = modestr[i];

    if (c == '+' || c == '-')
    {
        sign = c;
        continue;
    }
```

Every character is either a sign (`+`/`-`, update `sign` and move on) or a flag letter (handled below).

The `i` flag — invite-only

```
cpp
if (c == 'i')
{
    channel->set_invite_only(sign == '+');
    applied = true;
}
```

No argument needed. `sign == '+'` evaluates to `true`/`false` directly, so `+i` sets it on, `-i` sets it off, in one line.

Where this is enforced: NOT in `mode.cpp` at all — in `handl_join` (`server_cmds_channel.cpp`). `MODE` only flips the flag; `JOIN` is what actually checks it:

```
cpp
if (existing->is_invite_only() && !existing->is_invited(fd))
{
    send_numeric_reply(fd, 473, ..., ":Cannot join channel (+i)");
    return;
}
```

This is a common point of confusion: setting a mode and enforcing a mode are two different responsibilities, living in two different handlers.

The `t` flag — topic restricted to operators

```
cpp
```

```

else if (c == 't')
{
    channel->set_topic_restricted(sign == '+');
    applied = true;
}

```

Same pattern, no argument. **Enforced in** `handle_topic` (`server_channel2.cpp`), and only for the *setting* path, never the *querying* path:

```

cpp
if (channel->is_topic_restricted() && !channel->is_operator(fd))
{
    send_numeric_reply(fd, 482, ..., ":You're not channel operator");
    return;
}

```

This check must come AFTER the "no second parameter = just querying" check in `handle_topic`, for the same reason as Step 5 above.

The `k` flag — channel key (password)

```

cpp
else if (c == 'k')
{
    if (sign == '+')
    {
        if (arg_index >= msg.params.size())
        {
            send_numeric_reply(fd, 461, ..., "MODE :Not enough parameters");
            return "";
        }
        channel->set_key(msg.params[arg_index]);
        change_args.push_back(msg.params[arg_index]);
        arg_index++;
    }
    else
        channel->unset_key();
    applied = true;
}

```

Setting (`+k`) needs an argument — the new key/password. Bounds-checked before indexing `msg.params[arg_index]`, since a client can send `MODE #chan +k` with nothing after it, and indexing past the end of a `vector` is undefined behavior, not a safe out-of-range exception.

Unsetting (`-k`) needs NO argument — just clears it.

`change_args.push_back(...)` records the key so the broadcast line includes it, matching real IRC servers (though some servers mask the key as `*` in broadcasts for privacy — worth considering as a polish item,

not required for the mandatory scope).

Where this is enforced: `handl_join`, checking the key the joiner supplied (`msg.params[1]` in the JOIN command, i.e. `JOIN #chan <key>`) against the stored one:

```
cpp
if (existing->has_channel_key())
{
    if (msg.params.size() < 2 || msg.params[1] != existing->get_key())
    {
        send_numeric_reply(fd, 475, ..., ":Cannot join channel (+k)");
        return;
    }
}
```

The `o` flag — grant/revoke operator status

```
cpp
else if (c == 'o')
{
    if (arg_index >= msg.params.size())
    {
        send_numeric_reply(fd, 461, ..., "MODE :Not enough parameters");
        return "";
    }
    const std::string &target_nick = msg.params[arg_index];
    int target_fd = find_client_fd_by_nickname(target_nick);
    arg_index++;

    if (target_fd == -1 || !channel->is_member(target_fd))
    {
        send_numeric_reply(fd, 441, ...,
            target_nick + " " + channel_name + ":They aren't on that channel");
        return "";
    }
    if (sign == '+')
        channel->add_operator(target_fd);
    else
        channel->retirer_operator(target_fd);

    change_args.push_back(target_nick);
    applied = true;
}
```

Unlike `k` and `l`, **both** `+o` and `-o` need an argument (a nickname) — you always have to say WHO you're promoting/demoting. Steps:

Bounds-check before reading the nickname argument.

Resolve the nickname to an fd via `find_client_fd_by_nickname`.

Reject if that fd doesn't exist, or exists but isn't a member of THIS channel (you can't make someone an operator of a channel they're not even in).

Apply — `add_operator` or `retirer_operator` on the `Channel` object.

No separate enforcement needed elsewhere — operator status is checked live, at the moment of use, by every handler that cares (`KICK`, `INVITE`, `TOPIC` when `+t`, `MODE` itself). Because `channel->is_operator(fd)` always reflects the current state, a `-o` here immediately takes effect the next time that client tries an operator-only action — no extra propagation code needed.

The `1` flag — user limit

```
cpp
else if (c == '|')
{
    if (sign == '+')
    {
        if (arg_index >= msg.params.size())
        {
            send_numeric_reply(fd, 461, ..., "MODE :Not enough parameters");
            return "";
        }
        long limit = std::strtol(msg.params[arg_index].c_str(), NULL, 10);
        arg_index++;
        if (limit <= 0)
            continue;
        channel->set_user_limit(static_cast<int>(limit));
        change_args.push_back(msg.params[arg_index - 1]);
    }
    else
        channel->unset_user_limit();
    applied = true;
}
```

`+1` needs a numeric argument; `-1` needs none — mirrors `k`.

`std::strtol` converts the string to a number safely (it doesn't crash on non-numeric input the way `atoi` implicitly can be misused — it just returns 0, which the `limit <= 0` check then rejects).

`continue` (not `return ""`) on an invalid number: this means the bad `1` is silently skipped, but the loop keeps processing the rest of the mode string. That's a deliberate design choice — an isolated bad argument doesn't have to void the whole command. (Contrast with `k/o`'s missing-argument case, which does `return ""` — a genuinely missing argument is different from a present-but-invalid one, though reasonable implementations could go either way here; the important thing is being consistent and never crashing.)

Where this is enforced: `handl_join`, comparing current member count against the stored limit:

cpp

```
if (existing->has_user_limit()
    && existing->get_members().size() >= static_cast<size_t>(existing->get_user_limit()))
{
    send_numeric_reply(fd, 471, ..., ":Cannot join channel (+l)");
    return;
}
```

Unknown flag characters

 cpp

cpp

```
else
{
    std::string unknown(1, c);
    send_numeric_reply(fd, 472, ...,
        unknown + ":is unknown mode char to me");
    return "";
}
```

Anything that isn't `i t k o l +` or `-` is rejected with `472` and the whole function bails out immediately (`return ""`) — no partial state change gets broadcast for a malformed command. Note this is *stricter* than the `1`-with-bad-number case above (which `continue`s rather than returning) — an outright unrecognized character is treated as more serious than a recognized flag with a bad argument. Again, a defensible design choice as long as it's applied consistently.

Building the broadcast line

 cpp

cpp


```

if (applied)
{
    if (sign != last_sign_written)
    {
        change += sign;
        last_sign_written = sign;
    }
    change += c;
}

}

if (change.empty())
    return "";
std::string mode_line = ":" + client_prefix(fd) + " MODE " + channel_name + " " + change;
for (size_t i = 0; i < change_args.size(); i++)
    mode_line += " " + change_args[i];
mode_line += "\r\n";
return mode_line;
}

```

The `last_sign_written` tracking means the output only inserts a new `+`/`-` character when the sign actually flips — so `+o1` (both plus) produces `+o1`, not `+o+1`, matching how real IRC servers format combined mode strings. `change_args` (built earlier, one push per flag that needed an argument) get appended in the same order the flags were processed, keeping the broadcast line's arguments aligned with its flag letters.

PART 4 — SIGNAL HANDLING

Why signals matter here

Two separate signals matter for this project, for two very different reasons.

SIGINT / SIGQUIT — graceful shutdown (Ctrl+C, Ctrl+)

By default, these signals kill the process immediately — no destructors run, no cleanup code executes. Installing a handler changes that:



cpp

```
// main.cpp
volatile sig_atomic_t g_stop = 0;

void handle_signal(int sig)
{
    (void)sig;
    g_stop = 1;
}

int main(int argc, char **argv)
{
    signal(SIGINT, handle_signal);
    signal(SIGQUIT, handle_signal);
    ...
}
```

The handler does the absolute minimum: set a flag. This is deliberate — inside a signal handler, you can only safely touch a small set of "async-signal-safe" operations. `std::cout`, container access, memory allocation, and most library calls are NOT safe to call from a handler, because the signal can interrupt your program at ANY point, including mid-way through one of those exact operations, corrupting state. A `volatile sig_atomic_t` flag is specifically guaranteed safe to set and read across a signal boundary.

The main loop then has to actually check that flag:

```

❏
cpp
// server.cpp
void Server::run_serv()
{
    while (!g_stop)
    {
        int ret = poll(&poll_fds[0], poll_fds.size(), -1);
        if (ret == -1)
        {
            if (errno == EINTR)
                break;
            std::cerr << "Erreur: poll() echoue" << std::endl;
            break;
        }
    }
    ...
}
```

Two things happen together here:

`while (!g_stop)` — checked once per loop iteration, so if a signal arrives while we're busy processing clients, we still notice it soon.

`errno == EINTR` — when `poll()` is blocked waiting for activity and a signal arrives, the signal interrupts the syscall, `poll()` returns `-1`, and `errno` is set to `EINTR` ("interrupted"). This is NOT a real failure — without checking for it specifically, the code would print a scary "poll() failed" error message for perfectly normal Ctrl+C behavior.

The result: pressing Ctrl+C now cleanly breaks the loop, `run_serv` returns, `main` returns, and `Server`'s destructor runs — actually closing every open socket, rather than relying on the OS to clean up after an abrupt kill.

SIGPIPE — the crash you might never see coming

This one's different: it's not about graceful shutdown, it's about preventing an actual crash.

The scenario: your server has queued output for a client (say, another user's `PRIVMSG`), and calls `send()` to deliver it. But the receiving client's socket has already been closed — maybe they disconnected abruptly, maybe their network dropped. Writing to a closed socket raises `SIGPIPE`. **The default action for `SIGPIPE` is to terminate the entire process immediately** — not just fail that one `send()` call, not just disconnect that one client — kill the whole server, dropping every connected client at once.

The fix costs one line, with no downside:



cpp

```
signal(SIGPIPE, SIG_IGN);
```

`SIG_IGN` means "ignore this signal entirely." With it ignored, a `send()` to a dead socket just returns `-1` with `errno == EPIPE` instead of raising the signal — which `send_repons_to_client` already handles correctly:



cpp

```
ssize_t sent = send(fd, out.c_str(), out.size(), 0);
if (sent <= 0)
{
    disconnect_client(index); // already the correct recovery
    return (false);
}
```

So the recovery logic was already right — the only missing piece was stopping the signal from firing and skipping that recovery logic entirely.

Why this is easy to miss in casual testing: on `localhost`, a disconnect is usually detected by `poll()` (via `POLLHUP`/`POLLERR`) *before* your code ever gets a chance to `send()` to that fd — so the crash path is a race condition that your own machine's loopback network tends to lose. Over a real network, or under different timing, the race can go the other way, and `send()` fires before the disconnect is noticed. `signal(SIGPIPE, SIG_IGN)` removes the race entirely — it's one line, it's standard practice for virtually every network server ever written in C/C++, and it should always be present regardless of whether you've personally managed to trigger the crash in testing.

Where each piece lives, and why

File	What goes there	Why
<code>main.cpp</code>	<code>signal()</code> calls, <code>g_stop</code> definition, <code>handle_signal</code>	Signals must be installed once, at startup, before anything can go wrong
<code>server.hpp</code>	<code>extern volatile sig_atomic_t g_stop;</code> , <code>#include <csignal></code>	Lets <code>server.cpp</code> see the same variable <code>main.cpp</code> defines
<code>server.cpp</code>	<code>#include <cerrno></code> , the <code>while (!g_stop) / EINTR</code> check	This is the only file with the actual <code>poll()</code> loop — the only place that needs to check the flag
<code>channel.cpp</code> / <code>channel.hpp</code> and <code>mode.cpp</code>	need NO changes for signals — signal handling is a process/event-loop concern, entirely separate from channel state or MODE parsing logic.	

PART 5 — QUICK REFERENCE: ALL FIVE MODE FLAGS

Flag	Meaning	Arg on <code>+</code> ?	Arg on <code>-</code> ?	Set in <code>mode.cpp</code>	Enforced in
<code>i</code>	invite-only	no	no	<code>set_invite_only</code>	<code>handl_join</code> (473)
<code>t</code>	topic restricted to ops	no	no	<code>set_topic_restricted</code>	<code>handl_topic</code> (482)
<code>k</code>	channel key/password	yes	no	<code>set_key</code> / <code>unset_key</code>	<code>handl_join</code> (475)
<code>o</code>	operator grant/revoke	yes (nick)	yes (nick)	<code>add_operator</code> / <code>retirer_operator</code>	live, everywhere <code>is_operator</code> is checked
<code>l</code>	user limit	yes	no	<code>set_user_limit</code>	<code>handl_join</code> (471)

The pattern to remember: `mode.cpp` only ever *stores* the flag on the `Channel` object. It never enforces anything itself. Enforcement lives in whichever handler actually needs to check that flag at the moment of use — `JOIN` for `i`/`k`/`l`, `TOPIC` for `t`, and "everywhere" for `o` since operator status is just checked live via `is_operator(fd)` by every handler that cares.